DCMG Application-Specific Theory and Code Cross-Reference

This document specializes the generic dissipativity and data-driven results of `P6_Data_Driven_Co_Design.pdf` to the implemented DC microgrid (DCMG). It is intended for direct side-by-side use with the MATLAB code. It states the model, dimensions, sign conventions, controller transformations, data equations, and numerical conditions that must hold for each implementation to represent the theory.

Date: 2026-07-16

How to use this document

- Start with Sections 1-5 to fix the physical and mathematical conventions.
- Use Sections 6-10 while reviewing the four controller-design functions.
- Use Sections 11-13 while reviewing data generation and simulation.
- Use Section 14 as a function-by-function acceptance sheet.
- Cross-reference unresolved implementation changes by identifier in `IMPLEMENTATION_REVIEW_CHECKLIST.md`.
- Code line numbers refer to the reviewed code on the date above. If edits move a function, search for the function name and the listed variables.

Status words used below:

- **Theory contract:** a condition required for the paper result to apply.
- **Implemented:** the active code follows the stated application equation.
- **Diagnostic only:** useful verification that must not enter a direct data-driven synthesis as model knowledge.
- **Current caveat:** active code differs from, softens, or does not yet verify the theory contract.

1. Fast theory-to-code map

Application operation	Primary code	Theory object
Construct one DG model	`DG.updateModel()`	Continuous DCMG state equation
Construct physical network	`TransmissionLine`, `DCMicrogrid.buildConductance()`	Resistive graph Laplacian `YBar`
Assemble aggregate matrices	`DCMicrogrid.buildSystemMatrices()`	Block-diagonal plant and selectors
Design equilibrium	`DCMicrogrid.solveSteadyState()`	Voltage regulation and current sharing constraints
Local model-based design	`DG.designLocalXiDissipative()`	Continuous-time local Xi-dissipativity plus global-feasibility necessary condition
Global model-based co-design	`DCMicrogrid.codesign_MB_DRC()`	L2-gain dissipativity, VSC gain, and communication topology
Stabilizing baseline	`DCMicrogrid.design_MB_GSC()`	Continuous-time Lyapunov stabilization with physical gain structure
Build sampled data/QMI	`DCMicrogrid.loadDataMatrices()`, `compute_Qw_from_wtilde()`	`X`, `Xbar`, `U`, `Y`, `W`, `Qw`, and `QBar_w`
Local data-driven design	`DG.designLocalXiDissipative_DataDriven()`	Robust local Xi-dissipativity for every data-consistent model
Global data-driven co-design	`DCMicrogrid.codesign_DD_DRC()`	Robust global L2-gain/topology co-design
Simulate interconnected plant	`DCMicrogrid.dynamics()`, `DG.dynamics()`	Aggregate DCMG error dynamics plus optional nonlinear limiters
Reproduce paper plant	`support/createPaperDCMicrogrid()`	Fixed six-DG, seven-line benchmark

2. Symbols, dimensions, and stored forms

Let `N` be the number of DGs and let every DG have two states and two physical input channels.

Symbol	Size	DCMG meaning	MATLAB representation
x_i=[V_i;I_ti]	2 x 1	PCC voltage and converter internal current	DG.x
u_i=[I_i;V_ti]	2 x 1	Exported line current and VSC voltage command	Formed inside runtime/data code
w_i=[w_Ii;w_Vi]	2 x 1	Primitive disturbances in the two input channels	DG.noise.w row
A_i	2 x 2	Local continuous-time state matrix	DG.A
E_i	2 x 1	Current-channel matrix	DG.E
B_i	2 x 1	VSC-command matrix	DG.B
BBar_i=[E_i,B_i]	2 x 2	Full physical input matrix	DG.BBar
D_i=[1;0]	2 x 1	Voltage selector, D_i' x_i=V_i	Local literal D
Dbar_i=[0;1]	2 x 1	Current/VSC-row selector	Local literal DBar
x=[x_1;...;x_N]	2N x 1	Aggregate state	DCMicrogrid.getStateVector()
D=blkdiag(D_i)	2N x N	D' x is the bus-voltage vector	DCMicrogrid.D
Dbar=blkdiag(Dbar_i)	2N x N	Dbar' x is internal-current vector	DCMicrogrid.DBar
YBar	N x N	Physical conductance Laplacian	DCMicrogrid.YBar
K_Li	1 x 2	Implementable local VSC feedback row	DG.K
K_G	N x 2N	Implementable global VSC feedback rows	DCMicrogrid.K
K_i^full	2 x 2	Full local effective-input gain	[zeros(1,2);DG.K]
K^full	2N x 2N	Full effective global gain used in LMIs	A decision/recovered intermediate, not obj.K
commAdj(i,j)	scalar	Link j -> i selected from block K_G(i,j)	DCMicrogrid.commAdj

Important implementation distinction:

```
Theoretical/effective full input gain: 2N x 2N
    first row of each 2 x 2 block = physical line-current relation
    second row of each 2 x 2 block = designed VSC command relation

Runtime obj.K: N x 2N
    contains only the designed VSC command rows
```

The comments on DCMicrogrid.D and DBar currently state 2N x 2N, but the constructed matrices are correctly 2N x N. The identities used throughout the code require the latter dimensions.

3. Application-specific physical model

3.1 One DG

For a ZI load, the load current is

```
I_Li = V_i/R_Li + Ibar_i.
```

The implemented converter dynamics are

```
C_i dV_i/dt  = I_ti - V_i/R_Li - Ibar_i - I_i - w_Ii
L_i dI_ti/dt = -V_i - Rf_i I_ti + V_ti + w_Vi.
```

Equivalently

```
dx_i/dt = A_i x_i + E_i Ibar_i + BBar_i u_i + BBar_i w_i

A_i = [ -1/(R_Li C_i),   1/C_i
        -1/L_i,          -Rf_i/L_i ]

E_i = [-1/C_i; 0]
B_i = [0; 1/L_i]
BBar_i = [E_i, B_i].
```

Code cross-reference:

- DG.updateModel(), DG.m:94-103: **Implemented**.

- DG.dynamics(), DG.m:109-152: evaluates this model after adding feedback,

noise, and the active limiters.

- DG.iload_fun is configured but inactive in the current derivative.

Direct checks:

```
assert(norm(dg.A-[ -1/(dg.RL*dg.C), 1/dg.C; ...
                    -1/dg.L,       -dg.Rf/dg.L ],inf) < 1e-12)
assert(norm(dg.BBar-[dg.E,dg.B],inf) < 1e-12)
```

Related revisions: DG-01, DG-02, DG-03, DG-04, DG-05.

3.2 Resistive physical network

For a line oriented from DG i to DG j

```
I_ij = (V_i-V_j)/R_ij,       g_ij=1/R_ij.
```

The physical graph produces the symmetric Laplacian

```
YBar_ii = sum_j g_ij
YBar_ij = -g_ij for a physical line
YBar_ij = 0 otherwise.
```

Therefore the vector of current exported from the DG buses is

```
I_line = YBar V = YBar D' x.
```

The positive sign means current leaving a DG bus. Since E_i is negative in the voltage equation, positive exported current decreases V_i, as required.

Code cross-reference:

- TransmissionLine.current(), TransmissionLine.m:41-43: **Implemented**.
- DCMicrogrid.buildConductance(), DCMicrogrid.m:72-107: **Implemented** for

a simple graph with one line per node pair.

- DCMicrogrid.dynamics(), DCMicrogrid.m:690-713: computes YBar*D'*X.

Required network invariants:

```
assert(norm(net.YBar-net.YBar','fro') < tol)
assert(norm(net.YBar*ones(net.N,1),inf) < tol)
assert(min(eig((net.YBar+net.YBar')/2)) >= -tol)
assert(all(diag(net.YBar) >= 0))
```

Related revisions: LINE-01, LINE-02, SYS-01, SYS-02.

3.3 Aggregate continuous-time plant

DCMicrogrid.buildSystemMatrices() constructs

```
A     = blkdiag(A_i)             2N x 2N
E     = blkdiag(E_i)             2N x N
B     = blkdiag(B_i)             2N x N
BBar  = blkdiag(BBar_i)          2N x 2N
D     = blkdiag([1;0])           2N x N
Dbar  = blkdiag([0;1])           2N x N
wBar  = [Ibar_1;...;Ibar_N].
```

With VSC command vector V_t, the aggregate physical model is

```
dx/dt = (A + E YBar D')x + B V_t + E wBar + BBar w.
```

This identity is the simplest independent check of the simulator. The test files form Aphysical = A + E*YBar*D' explicitly.

4. Equilibrium and error coordinates

4.1 Equilibrium equations

Let x_s, I_s, and u_s denote equilibrium state, exported current, and VSC command. The application equations are

```
0   = A x_s + E I_s + B u_s + E wBar
I_s = YBar D' x_s.
```

The implemented operating specifications are

```
D' x_s    = diag(epsV) V_rated
Dbar' x_s = epsI I_rated
0.8 <= epsV_i <= 1.2
0 <= epsI <= 0.98.
```

DCMicrogrid.solveSteadyState(), DCMicrogrid.m:941-1080, implements these relations. Its active objective is

```
||epsV-1||_2^2 + epsI.
```

Thus it actively prefers small current utilization, not utilization near one. The paper's application statement also permits bounds on equilibrium exported current and VSC command; those bounds are not active.

Required post-solve residuals:

```
rPlant = net.A*net.x_s + net.E*net.ss.I_ss + ...
         net.B*net.u_s + net.E*net.wBar;
rLine = net.ss.I_ss-net.YBar*net.D'*net.x_s;
assert(norm(rPlant,inf) <= tol)
assert(norm(rLine,inf) <= tol)
```

Related revisions: SS-01, SS-02, SS-03.

4.2 Error dynamics and implemented controller

Define xtilde=x-x_s. The VSC command is

```
V_t = u_s + K_L xtilde + K_G xtilde,
```

where K_L is the aggregate embedding of the rows DG(i).K and K_G is net.K. The linear error dynamics are

```
dxtilde/dt = [A + E YBar D' + B(K_L+K_G)]xtilde + BBar w.
```

This equation is used directly by `test_paper_model_based_configuration()`. It excludes clipping and staged events and is therefore the preferred equation for validating a linear certificate.

The equivalent full-input representation used by the controller LMIs is

```
dxtilde/dt = [A + BBar(K_L^full+K_G^full)]xtilde + BBar w,

D' K_L^full = 0
D' K_G^full = YBar D'.
```

The two closed-loop matrices are equal when the physical gain constraints hold.

5. Dissipativity specialization used by the DCMG

5.1 Local IF-OFP certificate

The local output is the complete error state:

```
y_i = xtilde_i,       C_i=I_2.
```

The external local input is a two-dimensional additive state-channel input containing the global interconnection effect and disturbance. The local supply matrix is restricted to

```
Xi_i = [ -nu_i I_2,    0.5 I_2
          0.5 I_2,    -rho_i I_2 ].
```

The supply rate is

```
s_i = -nu_i ||utilde_i||^2 + utilde_i' y_i - rho_i ||y_i||^2.
```

For the global composition theorem used here:

```
Xi_i^11 > 0  <=>  nu_i < 0
Xi_i^22 < 0  <=>  rho_i > 0.
```

The transformed scalar used by both local design functions is

```
xBar11_i = (-x22_i)x11_i = -nu_i rho_i > 0,
x11_i=-nu_i, x22_i=-rho_i.
```

5.2 Global L2-gain certificate

The network performance output is the full state error, `z=xtilde`, and the network disturbance is the aggregate additive state-channel disturbance. The implemented desired global supply matrix is

```
Y = [ gamma^2 I_(2N),   0
      0,               -I_(2N) ].
```

If the hard global LMI is feasible, zero initial storage implies

```
integral ||z||^2 dt <= gamma^2 integral ||w||^2 dt
```

for the continuous-time linear model under the theorem assumptions. A solver success flag alone is not this certificate. The unsoftened LMI, sign conditions, and recovery identities must also pass numerically.

5.3 Universal certificate acceptance rule

For every symmetric matrix inequality M >= 0, evaluate

```
Msym = (value(M)+value(M)')/2;
minEig = min(eig(Msym));
```

Accept only if `minEig >= -certificateTolerance`, with a tolerance justified from solver residuals and scaling. Determinants of leading principal minors are not numerically reliable for these matrices.

6. Model-based local design

Function: `DG.designLocalXiDissipative()`, `DG.m:209-337`.

6.1 Decision variables and recovery

```
P          2 x 2 symmetric storage variable
L          2 x 2 transformed local gain, first row fixed to zero
xBar11     scalar > 0
x22        scalar < 0
x12=x21   0.5
KTilde, KHat and yBar variables: local proxy for global feasibility
```

The implemented recovery is

```
K_i^full = L P^(-1)
K_Li     = Dbar_i' K_i^full
rho_i    = -x22
nu_i     = -xBar11/(-x22).
```

Because the first row of `L` is zero and `P` is nonsingular, `D_i' K_i^full=0` is satisfied structurally.

6.2 Main continuous-time local LMI

After the paper's scalar-block transformation, the active `LMI1` is

```
[ I,       P,                          0
  P,      -H(A_i P+BBar_i L),          x22 I + 0.5 P
  0,       x22 I + 0.5 P,              xBar11 I ] >= epsilon I.
```

This is the continuous-time revised local dissipativity condition specialized to `C_i=I_2` and the DCMG two-input structure.

`LMI2` is the decomposed necessary condition intended to favor feasibility of the subsequent global co-design. Its active code form matches the paper's block structure. The equality that removes its relaxation is represented by

```
xBar11 KHat = x21 KTilde.
```

The code fixes the left scalar to `xBar11ValGuess=0.002` during optimization. The optimized `xBar11` can differ, so an exact unrelaxed interpretation needs iteration and a post-solve residual check.

6.3 Required acceptance checks

1. Solver status is zero before any `value()` or gain recovery.
2. `P`, `LMI1`, and `LMI2` meet the selected minimum-eigenvalue tolerance.
3. `xBar11>0`, `x22<0`, `nu<0`, and `rho>0` are finite.
4. `norm(D_i'*K_i^full)` is below tolerance.
5. `A_i+BBar_i*K_i^full` is Hurwitz for the continuous-time design.
6. If claiming the unrelaxed necessary condition, the BMI-surrogate residual

is below tolerance after iteration.

Related revisions: `MB-L01`, `MB-L02`, `MB-L03`, `NUM-02`.

7. Model-based global designs

7.1 Stabilizing baseline

Function: `DCMicrogrid.design_MB_GSC()`, `DCMicrogrid.m:1083-1148`.

The optimization uses `P=P'>0`, `L`, and

```
H(A P+BBar L) < 0
D' L = YBar D' P.
```

After recovery

```
K_full = L P^(-1)
D' K_full = YBar D'
K_G = Dbar' K_full.
```

Thus `A+BBar*K_full` equals `A+E*YBar*D'+B*K_G`. This is a useful equality to assert before simulation. When `useData=true`, the active code perturbs `K_G` elementwise by as much as 200 percent after optimization, invalidating the original stability certificate unless stability is rechecked.

Related revisions: `GSC-01`, `GSC-02`, `GSC-03`.

7.2 Hierarchical dissipative co-design

Function: `DCMicrogrid.codesign_MB_DRC()`, `DCMicrogrid.m:1186-1366`.

For each local certificate, construct

```
Xi11_i      = -nu_i I_2
Xi12_i      =  0.5 I_2
```

```
Xi22_i      = -rho_i I_2
Xi_p^kl     = blkdiag(p_i Xi_i^kl)
XiBar21_i   = (Xi11_i)^(-1)Xi12_i = -1/(2nu_i) I_2.
```

The code variable named `X_12` at this stage is the normalized block `XiBar12`, not the original `0.5 I_2` block.

The pre-scaled full effective gain is constructed blockwise as

```
Kpre_ij = [ p_i YBar_ij,   0
            KHat_i,j(1:2) ].
```

This construction enforces

```
D' Kpre = P YBar D'.
```

The VSC command gain recovered for runtime is

```
K_G = P^(-1) KHat.
```

The application-specific global LMI assembled in the code is

```
Luy   = Xi11 BBar Kpre
Dmat  = blkdiag(Xi_p^11,I)
Mmat  = [Luy, Xi_p^11; I, 0]
Theta = [-XiBar21 Luy-Luy' XiBar12-Xi_p^22, -Xi_p^21
         -Xi_p^12,                          gamma^2 I]
W     = [Dmat,Mmat;Mmat',Theta].
```

The theory contract is the hard condition `W>=0` with `P>0` and `gamma^2>=0`. The active implementation instead enforces

```
W-epsilonSlack >= epsilon I,
-I <= epsilonSlack <= I.
```

A negative diagonal slack makes the condition easier than the theorem LMI. Therefore, a run with `sol.problem==0` but `minEig(W)<-tol` is not a certified paper-theory solution.

After communication thresholding, re-evaluate the physical identity, closed-loop poles, and the hard dissipativity LMI using the pruned gain.

Related revisions: `MB-G01`, `MB-G02`, `MB-G03`, `MB-G04`, `NUM-01`.

8. Sampled-data contract for direct data-driven design

8.1 Required one-step equation

For every DG and every sample interval, the data-driven theory requires one consistent discrete-time equation

```
x_i[k+1] = A_di x_i[k] + B_di u_i[k] + w_di[k]
y_i[k]   = C_i x_i[k],              C_i=I_2.
```

For exact zero-order hold of the continuous local model

```
A_di = expm(A_i Ts)
B_di = integral_0^Ts expm(A_i tau) BBar_i dtau.
```

The sample input must be the complete deviation input

```
utilde_i[k] = [ I_i[k]-I_si
                V_ti[k]-V_tsi ],
```

and both components must be held or otherwise integrated consistently over the same interval. The additive `w_di[k]` is the interval effect in the state equation, not necessarily the primitive instantaneous noise sample.

8.2 Data matrices and required rank

With T transitions

```
X_i     = [x_i[0],...,x_i[T-1]]       2 x T
Xbar_i  = [x_i[1],...,x_i[T]]         2 x T
U_i     = [u_i[0],...,u_i[T-1]]       2 x T
Y_i     = X_i                         2 x T
W_i     = [w_i[0],...,w_i[T-1]]       2 x T.
```

The data-richness requirement is

```
rank([U_i;X_i]) = 4.
```

Report both the numerical rank and the smallest singular value. The first input row is the physically generated line current and is not independently actuated, so this rank condition cannot be assumed from VSC excitation alone.

8.3 Current active data construction

`DCMicrogrid.loadDataMatrices()`, `DCMicrogrid.m:449-554`, currently:

1. fixes `Ts=1e-5`;
2. linearly interpolates state samples;
3. uses previous-value interpolation for inputs;

4. sets `Y_i=X_i`;

5. defines the disturbance residual using forward Euler and the known model:

```
W_i = Xbar_i - [(I+Ts A_i)X_i + Ts BBar_i U_i].
```

This is useful as a simulation diagnostic, but it is not model-free and is not the exact-ZOH equation unless the approximation error is deliberately included in the disturbance set. Stages 1 and 5 are integrated continuously without holding every data input, which further weakens the exact one-step contract.

Related revisions: `DATA-01`, `DATA-02`, `DATA-03`, `DD-L05`, `DD-L06`.

9. Disturbance QMI and data-consistent model set

9.1 Local disturbance bound

For `W_i` of size `2 x T`, the required assumption is

```
[I_2;W_i']' Q_wi [I_2;W_i'] >= 0,
Q_wi=Q_wi', and Q_wi^22<0.
```

`Q_wi` has size `(2+T) x (2+T)`. In the implemented structured choice

```
Q_wi = [Q_I, 0; 0,-I_T],
```

so the condition reduces to `Q_I-W_i W_i'>=0`.

`compute_Qw_from_wtilde()` fits `Q_I` to the same realized residual used for design. This can describe that realization, but the robust claim requires a valid a-priori admissible disturbance family and the matrix S-lemma regularity condition. A reserve margin fitted after observing the data is weaker evidence than a physically justified bound selected before collection.

9.2 QMI on unknown local model parameters

Define

```
L_i = [ I_2, Xbar_i
        0,  -X_i
        0,  -U_i ]                  6 x (2+T)

QBar_wi = L_i Q_wi L_i'             6 x 6.
```

Then every admissible `(A_di,B_di)` must satisfy

```
[I_2;A_di';B_di']' QBar_wi [I_2;A_di';B_di'] >= 0.
```

This construction is implemented in `loadDataMatrices()`. For synthesis, normalizing a QMI by a positive scalar is permissible only if the same scaling is carried consistently into every local and aggregate use. Independent local normalization followed by a new aggregate normalization changes relative block weights under a single global S-lemma multiplier.

Related revisions: `DATA-04`, `DD-L02`, `DD-G02`.

10. Data-driven controller designs

10.1 Local robust design

Function: `DG.designLocalXiDissipative_DataDriven()`, `DG.m:340-609`.

The active function implements the revised data-driven local result. Key variables are

```
KHat       T x 2
Pbar       = X_i KHat                 2 x 2, symmetric positive
KTilde     2 x 2, first row zero
lambda1,lambda2 >= 0
xBar11>0, x22<0
proxy variables for the global necessary condition.
```

The recovered full local gain and implementable row are

```
K_i^full = KTilde (X_i KHat)^(-1)
K_Li     = Dbar_i' K_i^full.
```

`LMI1` is the matrix-S-lemma robustification of the local dissipativity condition. `LMI2` is the robustification of the relaxed necessary condition for the future global design. The two multipliers correspond to two QMI implications.

Required acceptance checks:

1. `rank([U_i;X_i])=4` and adequate smallest singular value.
2. `QBar_wi` has the intended sign/regularity and exact stored scale.
3. `X_i*KHat` is symmetric positive definite and well-conditioned.
4. Both S-lemma multipliers are nonnegative.

5. `xBar11>0`, `x22<0`, `nu<0`, and `rho>0`.
6. The unsoftened local LMIs meet minimum-eigenvalue tolerance.
7. The first row of the recovered full gain is zero.
8. The exact sampled closed loop is Schur stable as a diagnostic against the

known simulation plant. This validation may use the model; synthesis may not.

Current caveats include model-derived residual data, inconsistent QMI scaling, an inactive BMI relation, forward-Euler pole diagnostics, and post-processing before solver-success checks.

Related revisions: `DD-L01` through `DD-L06`, `NUM-02`.

10.2 Global robust co-design

Function: `DCMicrogrid.codesign_DD_DRC()`, `DCMicrogrid.m:1369-1659`.

Application dimensions are

```
QBar_w = blkdiag(QBar_wi)            6N x 6N
E_perm                               6N x 6N
Kpre                                 2N x 2N
P                                    N x N diagonal
Q                                    4N x 4N
lambda                               scalar >= 0.
```

`E_perm` changes the ordering of the block-diagonal local model descriptions to the aggregate ordering required by the global QMI. Because this application has `n_xi=n_ui=2`, all three local parameter groups have equal block size.

The global desired supply is `Y11=gamma^2_I, Y22=-I`. The code builds the paper's `Q`, `S`, and `R` blocks and uses

```
R = R1-lambda*EQ_wMat
W = [Q,S;S',R].
```

The physical affine constraint is

```
D' Kpre = P YBar D'.
```

After solution, the runtime VSC command gain is

```
K_G = P^(-1) Dbar' Kpre.
```

The theory contract requires `Q>0`, `P>0`, `lambda>=0`, `gamma^2>0`, the hard unsoftened `W>=0`, and a small physical-equality residual. The active code uses a diagonal soft slack, rescales the physical equality by `1e6`, and forces `epsilon=gamma^2=1e-3` through opposing bounds. These choices must not be treated as an implementation of the hard certificate without the independent post-solve checks.

Related revisions: `DD-G01` through `DD-G07`, `NUM-01`.

11. Communication topology and gain pruning

`buildCommAdjFromK()` partitions runtime `K_G` into `1 x 2` blocks. It retains block `(i,j)` when

```
||K_G,ij||_2 >= threshold * max(abs(K_G(:))).
```

The resulting adjacency is directed: an active block means DG `i` needs the state of DG `j`, or a communication link `j -> i`. The function comment calls the graph undirected, but no symmetrization is performed.

Pruning is a controller modification performed after synthesis. Therefore:

1. store the unpruned and pruned gains separately;
2. handle an all-zero gain before relative thresholding;
3. recompute the physical residual;
4. recheck Hurwitz/Schur stability;
5. re-evaluate the applicable hard dissipativity certificate.

The communication cost in the co-design functions weights each VSC gain block by Euclidean DG distance. Self-blocks have zero distance and therefore no communication penalty. Physical line-current rows should not be interpreted as communication links.

12. Runtime simulation versus certified linear model

12.1 Continuous and ZOH operation

For model-based runs, `DCMicrogrid.dynamics()` recomputes global and local feedback continuously inside `ode45`. For data-driven operation, `simulateStageZOH()` samples both gains every `Ts`, stores `u_G_Hold` and `u_L_Hold`, and integrates the continuous plant between samples.

The exact sampled closed-loop validation should use the same hold convention. Forward-Euler eigenvalues are not a substitute for the exact ZOH spectral radius when validating this implementation.

12.2 Active nonlinearities

`DG_dynamics()` and `computeUWTrajectories()` actively clip

```
I_line_i to [-2 Irated_i, 2 Irated_i]
V_ti     to [-2 Vrated_i, 2 Vrated_i].
```

Independent clipping of `YBar*V` at each DG can violate KCL because the physical line-network current has already been determined by the common voltage vector. Both clips also make the runtime model nonlinear and outside the linear LMI certificate. Use two explicitly separated modes:

- **Certificate-validation mode:** no clipping, no pruning without

recertification, no parameter events, and a linear disturbance model.

- **Nonlinear stress mode:** limiters and events active, with activation counts

reported and no claim that the linear certificate directly covers them.

12.3 Current seven-stage event sequence

The active code performs the following sequence over the simulation horizon:

Stage	Interval	Active preparation/event
1	0-5%	For DD, install perturbed MB-GSC; collect continuous trajectory data
2	5-20%	First DD local/global design and ZOH operation
3	20-40%	Apply an instantaneous state perturbation
4	40-60%	Amplify noise only over the first 2% of the stage as currently indexed
5	60-65%	Perturb arbitrary entries of `A` and `Ibar`; redesign equilibrium; collect data
6	65-80%	Redesign DD controller and operate under ZOH
7	80-100%	Apply state perturbation and a brief amplified-noise interval

The event labels in comments do not always align with the actual preparation block. Event times also assume `tspan(1)=0`. The parameter event should perturb physical `L`, `C`, `Rf`, and `RL` and call `updateModel()` if the experiment is intended to represent component uncertainty.

Related revisions: `SIM-01` through `SIM-06`, `DATA-05` through `DATA-08`.

13. Paper benchmark and current main workflow

13.1 Recovered paper configuration

`support/createPaperDCMicrogrid()` reconstructs the model-based configuration used for the current paper figures from Git revision `6a0091d` and `archive/data/matlab.mat`.

```
N=6 DGs, M=7 physical lines, horizon=0.05 s
physical edges: (1,3), (2,3), (2,4), (3,4), (4,5), (4,6), (5,6)
line R [Ohm]: 1.0271409, 0.8081504, 0.9708764, 0.8193491,
              0.6876191, 0.7395164, 1.0589790
```

DG parameter vectors are stored at full precision in `support/createPaperDCMicrogrid.m:29-49`. The accepted archived dissipative controller gains and 15-link directed communication adjacency are stored at `support/createPaperDCMicrogrid.m:100-116`.

The recovered equilibrium has common internal-current utilization about `0.52147` and bus voltages approximately

```
[46.21,46.14,45.95,46.61,47.10,47.68] V.
```

`tests/test_paper_model_based_configuration.m` compares current synthesis with the archived accepted gains using the unclipped linear error model. At the last verified run:

Method	Status	Off-diagonal directed links	Max real pole
Current MB stabilizing	PASS	30	-649.28
Archived paper MB dissipative	PASS	15	-211.90
Current MB dissipative	WARN	0	-191.10

The current dissipative result is stable but does not reproduce the archived topology and uses the active softened global LMI.

13.2 Current `main.mlx`

The current main workflow uses

```
rng(7), N=4, horizon=0.2 s, output/noise step=1e-5,
noise factor=0.01 I_2, relative link threshold=1e-4.
```

It generates a new random plant through `getARandomDCMicrogrid()`. It is a later simplified experiment and is not the fixed six-DG paper configuration. Use the paper benchmark test for regression while revising controller functions.

14. Function-by-function acceptance sheet

`DG.updateModel()`

- Inputs: positive physical `L,C,RL`, nonnegative `Rf`.
- Must produce: the matrices in Section 3.1.
- Accept when: exact matrix identities and physical-parameter validation pass.

`TransmissionLine.current()` **and** `buildConductance()`

- Inputs: valid distinct endpoints and positive finite resistance.
- Must produce: oriented line current and a symmetric PSD Laplacian with zero row sums.
- Accept when: Section 3.2 invariants pass and parallel-line policy is explicit.

`buildSystemMatrices()`

- Inputs: all DG models already updated.
- Must produce: aggregate dimensions and block identities in Section 3.3.
- Accept when: `BBar=[E,B]`, selectors have size `2N x N`, and `wBar` matches every `Ibar_i`.

`solveSteadyState()`

- Inputs: assembled plant, physical Laplacian, ratings, and explicit bounds.
- Must produce: feasible `x_s`, `u_s`, and `I_s` satisfying Section 4.1.
- Accept when: solver success, state/input specifications, and both residuals pass. The objective must represent the intended sharing target.

`designLocalXiDissipative()`

- Inputs: one known continuous DG model.
- Must produce: local VSC gain, `nu<0`, `rho>0`, and valid local LMIs.
- Accept when: all six checks in Section 6.3 pass.

`design_MB_GSC()`

- Inputs: aggregate known model and physical topology.
- Must produce: a Hurwitz full effective gain satisfying the physical row.
- Accept when: Lyapunov LMI, physical residual, recovered closed-loop poles, and post-pruning poles pass without post-solve gain corruption.

`codesign_MB_DRC()`

- Inputs: successful local certificates from every DG.
- Must produce: hard global L2-gain certificate, runtime VSC gain, and topology.
- Accept when: local signs, hard `W`, `P`, `gamma^2`, physical residual, and post-pruning recertification pass.

`loadDataMatrices()`

- Inputs: one consistently sampled/Held trajectory and documented `Ts`.
- Must produce: exact data matrices in Section 8.2 and a justified `QBar_wi`.
- Accept when: one-step residual follows the chosen exact model, rank/singular values pass, and no unknown model matrix is used as a synthesis input.

`compute_Qw_from_wtilde()`

- Inputs: a disturbance model or validation realization.
- Must produce: symmetric `Q_w` with negative lower-right block and a valid bound.
- Accept for synthesis when: the bound is selected independently of the design realization, includes a justified margin, and satisfies S-lemma regularity.

`designLocalXiDissipative_DataDriven()`

- Inputs: valid `X,Xbar,U,Y,QBar_w`, full row rank, consistent scaling.
- Must produce: one gain robust for every model in the data-consistent set.
- Accept when: all eight checks in Section 10.1 pass.

`codesign_DD_DRC()`

- Inputs: successful DD local certificates and consistently assembled aggregate disturbance QMI.
- Must produce: robust hard global certificate, physical gain, and topology.
- Accept when: `Q>0`, `P>0`, `lambda>=0`, `gamma^2>0`, hard `W`, physical residual, and post-pruning validation pass with no model knowledge in the synthesis path.

`dynamics()`, `simulateStageZOH()`, and `simulate()`

- Inputs: controller mode, noise model, event protocol, `Ts`, and ODE settings.
- Must produce: trajectories from the same model claimed by the experiment.
- Accept when: linear certificate mode and nonlinear stress mode are separated, ZOH timing is consistent, and all event/limiter activations are recorded.

15. Minimal numerical record for an accepted controller

For each local design, retain:

```
solver status; nu; rho; full and VSC-only gains;
min eig(P); min eig(local LMI1); min eig(local LMI2);
physical-row residual; Hurwitz/Schur margin;
data rank, singular values, QMI scale, lambdas (DD only).
```

For each global design, retain:

```
solver status; gamma^2; P; lambda (DD only);
unpruned and pruned gains; directed adjacency; communication cost;
min eig(Q); min eig(hard W); soft-slack values if diagnostic;
physical equality residual; closed-loop spectral margin;
equilibrium residuals; limiter activation counts.
```

For every reported trajectory, also retain the complete plant parameters, random seed, `Ts`, noise interpretation, ODE tolerances, event schedule, and the exact controller/result record above. This is the evidence needed to connect a plot to the application-specific theory rather than only to solver success.